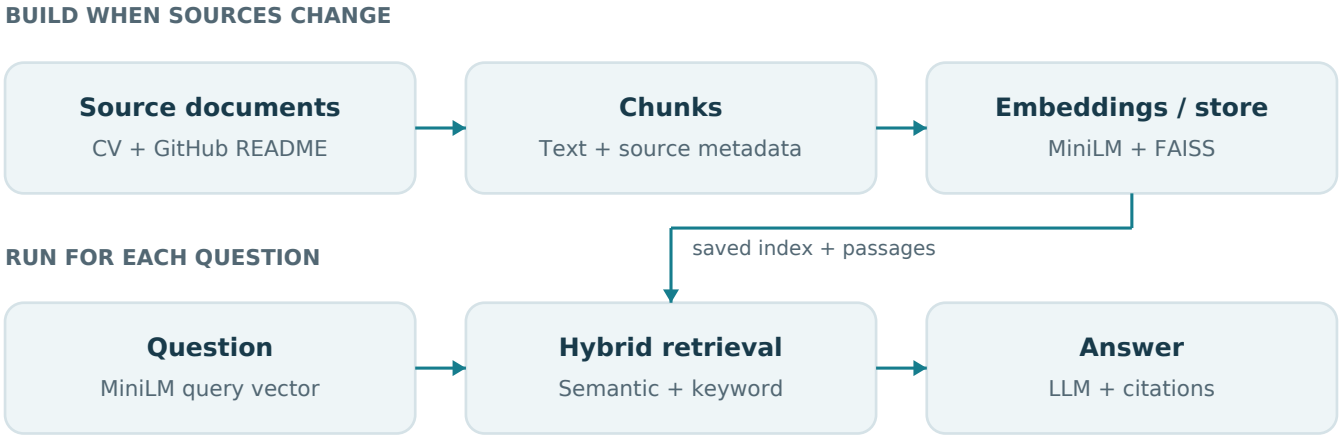
Task3_Task1
Personal RAG Knowledge Base

Implementation steps, design decisions, problems and lessons

The project turns a CV and a GitHub README into a searchable knowledge base about Nafe Abubaker. A question retrieves relevant passages; an answer model then uses those passages and identifies its sources. The completed application includes a command-line workflow and a deployed Streamlit interface.

Live application: cellula-tech-task-3.streamlit.app



Stored chunks	Embedding dimensions	Longest final chunk
19: 5 CV + 14 README	384 per chunk	221 tokens; limit 256

What RAG means in this project

Retrieval-Augmented Generation supplies an existing language model with selected evidence at question time. We built an index; we did not train a new model on the CV. Updating the knowledge base means rebuilding the saved chunks and vectors, rather than changing model weights.

Scope

Task 1 uses MiniLM for embeddings, FAISS for vector search, and a separate answer model through OpenRouter. BigBird belongs to the Task 0 research report and is not the model used by this application.

This report describes the implemented scripts and the development runs shared during the project. The manual checks demonstrate specific behaviors; they are not an accuracy benchmark.

1. Load sources and create chunks

Steps 1-3: turn source material into passages that can be retrieved and cited.

Step 1 - Extract the CV with pypdf

File: load_documents.py. We extracted text page by page, removed empty lines, and retained the filename, page number and PDF hyperlink annotations. The initial CV contained one page, 3,890 extracted characters and nine hyperlinks.

Why: an answer needs traceable evidence, not just anonymous text. Keeping page-level metadata lets the application show where a passage came from. Hyperlink annotations are separate from visible PDF text, so they needed to be captured separately.

Step 2 - Add the GitHub project README

File: load_github_readme.py. The GitHub API supplied the README of Abubaker-Nafe/ibt-ggateway-capstone. We decoded its content, saved a local copy, and added a second document with the repository name, original URL and retrieval metadata.

Why: the CV summarized the capstone, while the README contained project-specific evidence, including the team-role table. Refreshing the same README replaces its earlier loaded entry rather than duplicating it. Stored links are identifiers; the loader does not crawl every linked page.

Step 3 - Split by the embedding model's tokenizer

File: chunk_documents.py. The first splitter used 800 characters with a target overlap of 120 characters. After adding the README, one chunk contained 387 tokens, exceeding the embedding model's 256-token limit. A character limit could not control this reliably.

```
splitter = RecursiveCharacterTextSplitter.from_huggingface_tokenizer(
    tokenizer, chunk_size=220, chunk_overlap=30
)
```

The revised splitter measured length with MiniLM's tokenizer. Every final chunk was then counted again, including special tokens, and rejected if it exceeded 256. The resulting 19 chunks had a maximum length of 221 tokens. This check prevented silently discarding source content through truncation.

> **Tokens, chunks and overlap**
> A token is a unit produced by a tokenizer; it is not always a whole word. A chunk is a retrieved text passage. Overlap carries some boundary context into the next chunk, but costs duplicate storage. Our 220-token size and 30-token overlap are targets; recursive separators can produce shorter passages.

2. Embed passages and build the index

Steps 4-5: represent meaning numerically, then keep each vector connected to its source.

Step 4 - Create embeddings with MiniLM

File: create_embeddings.py. We used the pretrained sentence-transformers/all-MiniLM-L6-v2 model on CPU. It converted each passage into 384 numbers. The saved embedding matrix had shape **(19, 384)**: one row per chunk and one column per embedding dimension.

Why: an embedding lets retrieval compare meaning even when a question and a source use different wording. The 384 dimensions are learned numerical features, not a hand-written list of 384 skills. The same embedding model is used for stored passages and questions so their vectors can be compared.

```
vectors = model.encode(
    texts, convert_to_numpy=True, normalize_embeddings=True
).astype(np.float32)
```

Before embedding, we checked chunk token lengths again. After embedding, we checked row counts and finite values. We saved both the NumPy matrix and an ordered chunk list, preserving the mapping between row i and source passage i.

Step 5 - Store normalized vectors in FAISS

File: build_vector_store.py. We added the vectors to **IndexFlatIP** and saved the index alongside JSON metadata containing the text and source mapping. FAISS supplies a vector index; the JSON file supplies the readable passages and provenance.

Why: this exact index is simple and suitable for a collection of only 19 passages. It compares a query with every stored vector rather than using an approximate search structure. A separate hosted vector-database service was unnecessary for this scale.

```
index = faiss.IndexFlatIP(384)
index.add(vectors)
```

Cosine similarity and normalization

Cosine similarity measures the angle between vectors. Normalization makes each vector have length one, so its dot product with another normalized vector equals cosine similarity. That is why inner-product search can implement cosine retrieval here.

```
cosine(q, x) = dot(q, x) / (length(q) * length(x))
If both lengths are 1: cosine(q, x) = dot(q, x)
```

> **Two models, two jobs**
> MiniLM encodes text for retrieval. The OpenRouter answer model writes the response from the retrieved text. The answer model does not search the FAISS index itself, and the application does not send it the stored embedding vectors as evidence.

3. Retrieve the right evidence

Step 6: combine semantic search with keyword matching after observing a retrieval failure.

Why semantic retrieval alone was insufficient

File: retrieve_documents.py. Initially, we retrieved the three passages with the highest cosine similarity. For the capstone primary/secondary-role question, this selected general project passages but missed the explicit team table. The model correctly returned insufficient evidence in the first run.

We tried **top-k = 8** to retrieve more passages. The role table was still absent, and a generated answer incorrectly inferred role categories from general CV descriptions. It also inferred a pronoun. More context did not resolve the missing evidence.

Add TF-IDF keyword matching

TF-IDF weights words by their occurrence in a passage and their rarity across the collection. It provides a lexical signal for names, role labels and other exact terms. We combined this with the semantic signal instead of choosing only one retrieval method.

```
S = max(semantic_cosine, 0) / max_positive_semantic
K = keyword_cosine / max_keyword_cosine
retrieval_score = 0.4 * S + 0.6 * K
```

The implementation evaluates all stored vectors in this small collection, computes TF-IDF similarity, then ranks by the combined score. If no keyword matches exist, it falls back to semantic ranking. The 40/60 weights were chosen to recover the observed table; they have not been established as optimal by a benchmark.

Retrieved passage	Semantic cosine	Hybrid score
README overview: chunk_006	0.5824	0.8711
Team table: chunk_007	0.2634	0.7809
README discussion: chunk_016	0.5710	0.5830

In the observed hybrid run, the team table became the second retrieved passage despite its lower semantic similarity. The resulting answer identified **Project Coordinator and Data Lead** as primary roles, and **EDA Lead and GitHub Lead** as secondary roles, citing that table as [2].

> **A retrieval score is not answer confidence**
> The hybrid score is scaled within the current query and collection. It orders passages; it does not prove that a passage answers the question. Even an unrelated AWS-certifications query can produce high ranking scores, so the generation stage must still assess the evidence.

4. Generate and validate an answer

Steps 7-8: supply context to the answer model and join retrieval with generation.

Build the prompt from retrieved passages

File: generate_answer.py. Each passage is labelled [1], [2] or [3], with its source, page where applicable, chunk identifier and text. The system instruction restricts the answer to this evidence, requires citations, and treats passage content as data rather than instructions.

Why: the model should answer the user's question from the stored material. The prompt explicitly preserves primary/secondary labels, discourages unsupported facts and pronouns, and permits an incomplete answer when only part of a question is supported. These instructions reduce errors but cannot guarantee factual correctness.

Use a machine-readable status

```
{"status": "answered",
 "answer": "Nafe developed RESTful APIs [1]."}

{"status": "insufficient_evidence", "answer": ""}
```

An answered response must contain text and at least one citation belonging to the retrieved reference set. An insufficient-evidence status returns the standard message without requiring a citation. This avoids treating a valid refusal to invent information as a parsing failure.

> **Grounding and citations**
> Grounding means tying an answer to supplied evidence. A citation label makes that evidence inspectable. Checking that [1] exists does not prove that [1] supports the claim; the application validates labels, while claim-level support still needs review.

Separate output syntax from evidence quality

```
ANSWER_FORMAT = {"type": "json_object"}
response = llm.bind(response_format=ANSWER_FORMAT).invoke(messages)
```

The final provider accepted JSON-object mode, but rejected JSON-Schema mode. JSON-object mode requests JSON syntax; it does not enforce our complete field schema. Python checks the status, answered text and citation labels after receiving the response.

Join the stages with ask_question.py

File: ask_question.py. One command retrieves passages, generates an answer, prints the sources, and saves retrieval_results.json and answer.json. The CLI stages remain independently runnable, which made it possible to inspect retrieval before involving the answer model.

The final observed model was **apodex/apodex-1.1-mini:free** through ChatOpenRouter, with temperature 0 and a 60,000 ms timeout. This records the working configuration; it is not a promise that the same free endpoint will always remain available.

5. Problems encountered and fixes

The observed failure, the change made, and the lesson from each incident.

Observed problem	How we solved it and why
Chunk exceeded the model limit 387 tokens versus a 256-token limit after adding the README.	Replaced character-length splitting with MiniLM-token-aware splitting (220/30), then checked final lengths including special tokens. Text length in characters cannot guarantee model input length.
Semantic retrieval missed the role table Top three contained general project passages.	Tried top-k = 8, then added TF-IDF to semantic ranking. Hybrid retrieval brought the explicit team table into the top three; increasing k alone had not recovered it.
Incorrect role interpretation An answer inferred primary/secondary roles and a pronoun.	Retrieved the explicit table and strengthened the prompt to preserve source labels and use the person's name. Retrieval needed to improve; prompting alone was not enough.
Missing-evidence answer failed citation validation AWS question had no supported answer.	Introduced answered/insufficient_evidence statuses. Require citations for factual answers, but accept the standard insufficient-evidence message without a citation.
Timeout used the wrong units The earlier timeout was too short.	Set ChatOpenRouter timeout to 60_000: its units are milliseconds, so this means 60 seconds. Timeout units depend on the client library.
Model returned a privacy classification "User Safety: unsafe" was not answer JSON.	Moved away from the random free router and added response/model diagnostics. The original response did not identify its model, so we could not establish why it classified the input.
Pinned free endpoint was unavailable gpt-oss-120b:free returned an API error.	Selected the Apodex free endpoint instead. A model page or catalog entry does not establish that a specific live request will succeed; availability needs runtime confirmation.
HTTP 400 hid a format mismatch The initial message only said "Provider returned error".	Saved the exception body in last_api_error.json, redacting the key. Novita's detailed response said json_schema was unsupported and json_object was accepted. Switching the format resolved this error.

Warnings and failures are different

The Hugging Face unauthenticated-request warning did not prevent the public model from loading. It was not the cause of the retrieval or generation failures. We investigated the explicit exception and raw response instead.

6. Add Streamlit and deploy

Step 9: provide a browser interface without rebuilding the knowledge base for every question.

Reuse the existing pipeline

File: app.py. The interface offers example questions, a question field, an answer area, expandable passages, original-source links and a JSON download. It calls the same retrieval and generation functions used by the CLI.

Why: the web interface makes the completed RAG system easier to demonstrate and share. Streamlit and deployment were added by choice; the original Task 1 requirement was the functioning personal RAG system.

Cache resources; keep answers in the session

We separated load_retrieval_resources() from retrieve_documents() so Streamlit could cache the FAISS index, chunk metadata and embedding model with st.cache_resource. Index/metadata file timestamps invalidate the cached resource after a rebuild.

A lock protects the shared embedding model during concurrent requests. Answers live in st.session_state, so each browser session keeps its own latest result. If a new request fails, the app clears the earlier result instead of displaying it as the new answer. Caching resources avoids repeatedly loading model weights; it does not skip retrieval for new questions.

Run locally

```
python -m pip install streamlit
python -m streamlit run app.py --server.fileWatcherType none
```

Deploy the saved runtime inputs

The completed app is hosted on **Streamlit Community Cloud**:
https://cellula-tech-task-3.streamlit.app/

The GitHub deployment needs app.py, its retrieval/generation modules, a requirements.txt file, and the prebuilt output/vector_store/index.faiss and metadata.json in the same relative locations. Visitors do not trigger document ingestion, chunk creation or index construction.

Keep local and deployed credentials separate

Locally, the existing .env supplies OPENROUTER_API_KEY. In Community Cloud, the key is entered through Streamlit Secrets. It does not need to be committed to GitHub; .env remains in .gitignore.

```
OPENROUTER_API_KEY = "your_key_here"
```

To publish updated source material, refresh the documents, rebuild chunks, embeddings and the vector store, then push the updated index and metadata. The hosted interface loads that saved state.

7. Results and current limitations

What the project demonstrated, and what those checks do not establish.

Manual question	Observed behavior
Backend development experience	Returned a cited answer from CV passages. The final JSON-object configuration passed this live CLI check.
Primary and secondary capstone roles	Hybrid retrieval recovered the explicit team table and produced the correct role categories during retrieval development.
AWS certifications	Returned the insufficient-information message with the final generation model, instead of inventing certifications.

Streamlit interface checks covered source display, cached resources, session isolation and error handling with simulated embedding/answer responses. Those checks did not validate live model accuracy. Deployment was subsequently reported successful. There was no comprehensive test set, accuracy score, latency benchmark or load test.

Limited source coverage and freshness

The knowledge base contains one CV and one repository README. LinkedIn, portfolio pages, other repository files and hyperlink destinations are not automatically ingested. Saved snapshots can become stale. Scanned PDFs need OCR, which this loader does not implement.

Retrieval and chunk boundaries can lose evidence

A relevant passage can still rank below top-k. Small chunks can separate a role heading from its details, and token overlap does not guarantee that every table remains intact. The current 40/60 ranking weights were tuned to one observed failure, not evaluated across a broad set of questions.

Validation is limited to structure and labels

A syntactically valid, cited answer can still overstate its source. The parser does not verify every claim, detect contradictions or measure answer confidence. Insufficient evidence means the retrieved passages did not answer the question; it is not proof that a qualification does not exist.

Service and deployment constraints

Free providers can change models, supported formats, rate limits or availability. Cold starts and CPU model loading can add delay. The app has no authentication or per-user request limit; visitors share the owner's API quota. The model lock also serializes embedding work during concurrent requests.

Saved outputs are snapshots

CLI result files and diagnostic files are overwritten; they are not a conversation database. A failed CLI generation can leave an older successful answer.json. Streamlit answers are separate per session, but generator diagnostic files are shared and are not a durable request history.

8. Script map and technical references

A compact guide to the files, their responsibilities, and the concepts used in the report.

File	Responsibility
load_documents.py	Extract CV text and retain page/link metadata.
load_github_readme.py	Fetch and refresh the project README.
chunk_documents.py	Create token-aware passages with source identifiers.
create_embeddings.py	Create normalized MiniLM vectors and row mapping.
build_vector_store.py	Persist the FAISS index and matching metadata.
retrieve_documents.py	Rank passages using semantic and lexical evidence.
generate_answer.py	Build context, call the model and validate the response.
ask_question.py	Run retrieval and generation in one CLI command.
app.py	Serve the Streamlit interface and cache resources.

Run order

Load CV, add README, chunk, embed, then build the vector store. After that, use ask_question.py or app.py. A change to source text, chunk settings or embedding model requires a matching rebuild; mixing a new chunk list with an old index breaks the row-to-source mapping.

Technical references

1. **Sentence Transformers: all-MiniLM-L6-v2 model card**
Embedding dimensions, short-text encoding and default token window.

2. **LangChain: splitting by token**
Matching token counts to the model tokenizer.

3. **FAISS: MetricType and distances**
Normalized inner product and cosine similarity.

4. **scikit-learn: text feature extraction**
TF-IDF features for lexical matching.

5. **Streamlit: st.cache_resource**
Caching shared resources across reruns and sessions.

6. **Streamlit: Community Cloud secrets management**
Keeping deployed credentials outside the Git repository.

Implementation decisions, error messages and example results are drawn from the project scripts, saved metadata and shared development runs. Documentation supports the general concepts; it does not independently verify the project's generated answers.